

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _CH.NAGA SAI SRIKANTH(192311159) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

CH.NAGASAISRIKANTH

Signature of the student:

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

ANSWERS

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

ANS : Case Analysis

The software company uses MD5 to store user passwords. Attackers were able to crack several passwords after the database was compromised. This happens because MD5 is no longer considered secure for password storage.

Weaknesses of MD5

- MD5 is vulnerable to collision attacks.
- It is a **very fast hash function**, which allows attackers to try a huge number of password guesses quickly.
- MD5 does not provide protection against **rainbow-table attacks** when passwords are stored without unique salts.
- If identical passwords are stored using unsalted MD5, they produce identical hash values.
- A database breach exposes the password hashes, which can then be attacked offline.

Secure Solution

A password system should use a **salted, password-specific hashing/KDF algorithm** rather than plain MD5 or plain SHA-256.

A SHA-based approach can use **PBKDF2-HMAC-SHA-256** with:

- A unique random salt for every password.
- A sufficiently high iteration count.
- A cryptographically secure random salt.
- Secure password verification rather than storing plaintext passwords.

For example:

Conclusion

MD5 should be completely removed from password storage. A strong password KDF such as **PBKDF2-HMAC-SHA-256 with a unique salt** provides much better resistance to offline password-cracking attacks.

2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

Case Analysis

The banking application uses digital signatures to verify transactions, but unauthorized transactions are being approved. This indicates a possible failure in **signature verification, key management, transaction binding, or certificate validation**.

Possible Flaws

- The application may not correctly verify the digital signature.
- The sender's public key may not be properly authenticated.
- Private signing keys may have been compromised or poorly protected.
- The signature may not cover all important transaction information.
- The application may be vulnerable to a **replay attack** if transaction identifiers, timestamps, or nonces are not included.
- Weak or obsolete signature algorithms may be used.
- Certificate validation may be incorrectly implemented.
- Error handling may allow transactions to proceed when signature verification fails.

Recommended Improvements

- Use approved modern digital signature standards and algorithms.
- Use sufficiently strong **RSA-PSS, ECDSA, or EdDSA**, where supported by the system and applicable standards.
- Protect private keys using secure key storage such as an HSM or appropriate secure key-management system.
- Verify the complete transaction data, including:
 - Account information
 - Transaction amount
 - Transaction ID
 - Timestamp or nonce
- Reject a transaction whenever signature verification fails.

- Implement certificate validation and certificate-chain verification where certificates are used.
- Use secure key rotation and revocation procedures.
- Maintain audit logs for signature verification and transaction approval.

Conclusion

A digital signature is secure only when **key protection, signature verification, transaction binding, certificate validation, and replay protection** are correctly implemented. The banking application should use modern standardized signature algorithms and strict verification procedures.

3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

Case Analysis

Kerberos uses tickets and timestamps to authenticate users. A replay attack can occur when an attacker captures a valid authentication message or ticket and attempts to reuse it later.

How the Attack Could Occur

For example:

1. A legitimate client authenticates with the Kerberos system.
2. An attacker captures an authentication message or authenticator.
3. The attacker attempts to send the captured information again.
4. If the server does not properly check timestamps, ticket lifetime, or replay information, it may accept the old authentication data.
5. The attacker may then obtain unauthorized access.

Possible Causes

- Poorly synchronized clocks.
- Excessively long ticket or authenticator lifetimes.
- Incorrect timestamp validation.
- Failure to maintain replay-detection information.
- Compromised credentials or service keys.

- Incorrect Kerberos configuration.

Recommended Enhancements

- Ensure **clock synchronization** between clients and Kerberos servers.
- Strictly validate timestamps and ticket lifetimes.
- Use short-lived authenticators.
- Maintain replay caches where appropriate to detect reused authenticators.
- Protect Kerberos keys and credentials securely.
- Follow secure Kerberos configuration practices.
- Use strong authentication and monitor suspicious authentication attempts.
- Immediately revoke or replace compromised credentials and keys.

Conclusion

Kerberos is specifically designed with **timestamps and authenticators** to reduce replay attacks. Correct time synchronization, strict ticket validation, replay detection, and secure key management are essential for preventing such attacks.

4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

Case Analysis

The communication system uses only a simple hash function to verify message integrity. Attackers can modify messages without detection because a hash by itself does not prove that the hash was generated by a trusted sender.

Why Simple Hashing Fails

Suppose the sender sends:

An attacker modifies the message:

The attacker can also calculate:

and send:

The receiver calculates the hash of , obtains the same value, and may incorrectly assume that the message is authentic.

Therefore, a normal hash provides **integrity checking but not authentication against an active attacker who can replace both the message and its hash.**

HMAC Solution

HMAC combines a secret key with a cryptographic hash function.

For example:

where:

- = secret key
- = message

The sender sends:

An attacker who does not know the secret key cannot normally generate a valid HMAC for a modified message.

Recommended Approach

Use a standardized HMAC construction such as:

with:

- A strong randomly generated secret key.
- Secure key storage.
- Secure key distribution.
- Proper key rotation.
- Constant-time verification where appropriate.
- Rejection of messages whose HMAC verification fails.

Conclusion

A simple hash cannot authenticate the source of a message. **HMAC-SHA-256** provides both **message integrity and message authentication**, assuming the secret key remains secure.

5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

Case Analysis

The organization uses X.509 certificates, but users frequently receive trust-validation errors. This usually indicates problems with **certificate configuration, certificate chains, expiration, trust stores, or certificate validation.**

Possible Issues

1. **Expired certificate**
The certificate's validity period may have ended.
2. **Certificate chain problem**
The server may not provide the required intermediate CA certificates.
3. **Untrusted Certificate Authority**
The root CA may not be present in the client's trusted root store.
4. **Hostname mismatch**
The domain name being accessed may not match the certificate's valid identity, such as the Subject Alternative Name (SAN).
5. **Revoked certificate**
The certificate may have been revoked by the CA.
6. **Incorrect system time**
An incorrect date or time can make a valid certificate appear expired or not yet valid.
7. **Improper certificate installation**
Certificates may be installed in the wrong location or with an incorrect configuration.
8. **Weak or unsupported certificate configuration**
Older algorithms, key sizes, or signature configurations may cause compatibility or security problems.

Recommended Solutions

- Obtain certificates from a **trusted Certificate Authority (CA)**.
- Install the complete certificate chain, including required intermediate certificates.
- Ensure trusted root CA certificates are correctly installed on client systems.
- Renew certificates before expiration.
- Verify the **Subject Alternative Name (SAN)** against the hostname.
- Check certificate revocation status using appropriate mechanisms.
- Synchronize system clocks using reliable time services.
- Protect CA and private keys securely.
- Maintain an inventory of certificates and their expiration dates.

- Automate certificate renewal where practical.
- Regularly audit the certificate chain and trust configuration.

Certificate Trust Chain

Root CA

|

v

Intermediate CA

|

v

Server Certificate

|

v

Client verifies certificate

Conclusion

X.509 trust errors are commonly caused by incorrect certificate chains, expired certificates, untrusted CAs, hostname mismatches, revocation problems, or incorrect system configuration. Proper **PKI management, certificate-chain validation, CA trust management, renewal, and monitoring** ensure reliable authentication and secure communication.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand